JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

FPGA Design Using VHDL

Module 5C: Shift Registers in VHDL

Shift Registers

- A **shift register** is a series of registers that shift a value on …
  - Every clock cycle or …
  - Every time an enable signal goes high
- Shift registers are described as R-deep and N-bits wide
  - Where did we use shift registers?

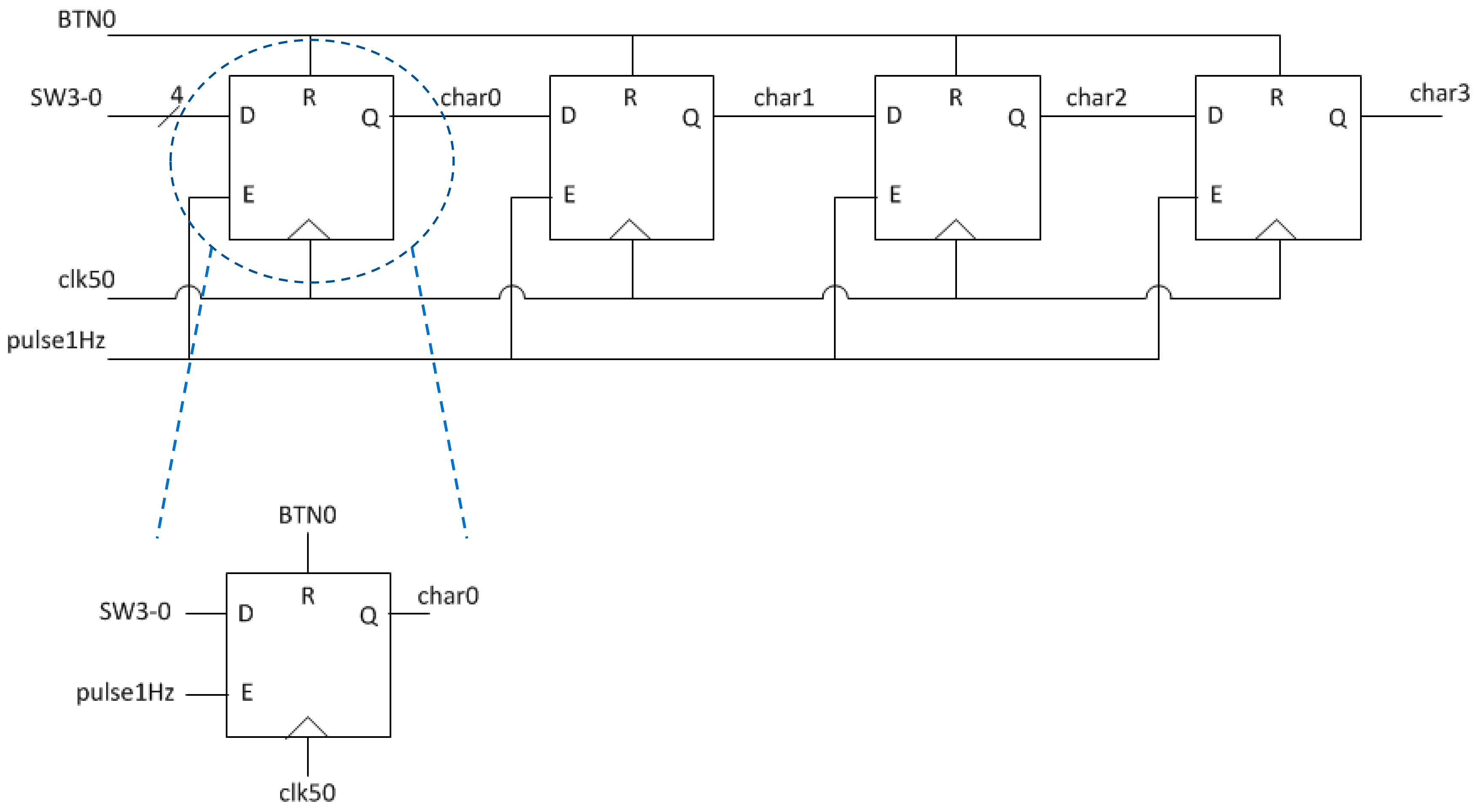
Shift Registers – Lab 3

- Lab 3 required a design that shifts characters at a rate of 1 Hz (scrolling affect)
- Each character is a 4-bit vector
- Stores eight characters since there are four characters to display

Shift Registers – Lab 3 Continued

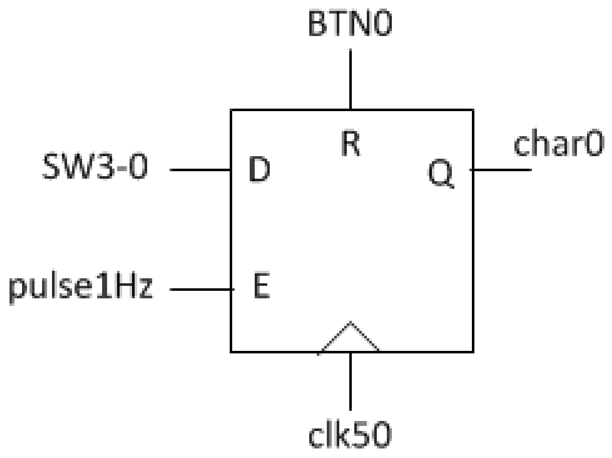
- In words:
  - We need to design a **4-bit, eight-deep** shift register that shifts values at 1 Hz
  - This can be implemented using four DFFs, each DFF is 4 bits
  - Triggered by the rising edge of the clock
  - Asynchronously reset
  - Enable signal, generated by the 1 Hz Pulse Generator

4-Bit, Four-Deep Shift Register

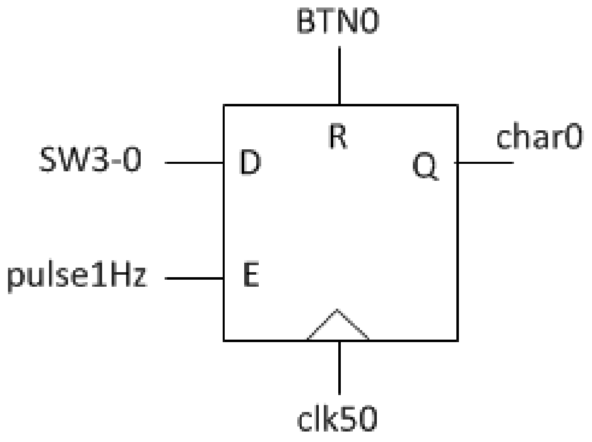


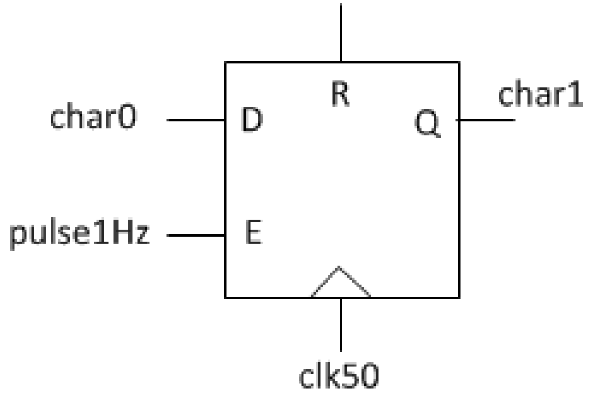
VHDL Code – One Process

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char0 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char0 <= SW(3 downto 0);
      end if;
   end if;
end process;
```



VHDL Code – Two Processes

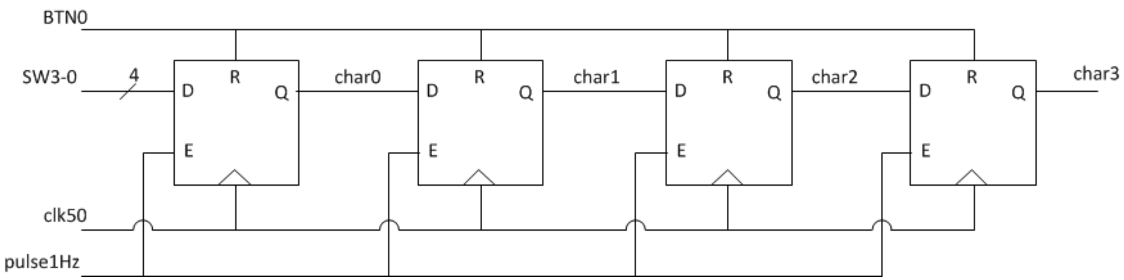





```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char0 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char0 <= SW(3 downto 0);
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char1 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char1 <= char0;
      end if;
   end if;
end process;
```

VHDL Code – All Four Processes




```
process(BTN0, clk50)
begin
   if(BTN0 = ‘1’) then
      char0 <= (others => ‘0’);
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = ‘1’) then
         char0 <= SW(3 downto 0);
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = ‘1’) then
      char1 <= (others => ‘0’);
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = ‘1’) then
         char1 <= char0;
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = ‘1’) then
      char2 <= (others => ‘0’);
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = ‘1’) then
         char2 <= char1;
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = ‘1’) then
      char1 <= (others => ‘0’);
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = ‘1’) then
         char3 <= char2;
      end if;
   end if;
end process;
```

VHDL Code – All Four Processes Continued

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char0 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char0 <= SW(3 downto 0);
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char1 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char1 <= char0;
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char2 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char2 <= char1;
      end if;
   end if;
end process;
```

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char1 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char3 <= char2;
      end if;
   end if;
end process;
```

VHDL Code - Compact

```
process(BTN0, clk50)
begin
   if(BTN0 = '1') then
      char0 <= (others => '0');
      char1 <= (others => '0');
      char2 <= (others => '0');
      char3 <= (others => '0');
   elsif(rising_edge(clk50)) then
      if(pulse1Hz = '1') then
         char0 <= SW(3 downto 0);
         char1 <= char0;
         char2 <= char1;
         char3 <= char2;
      end if;
   end if;
end process;
```

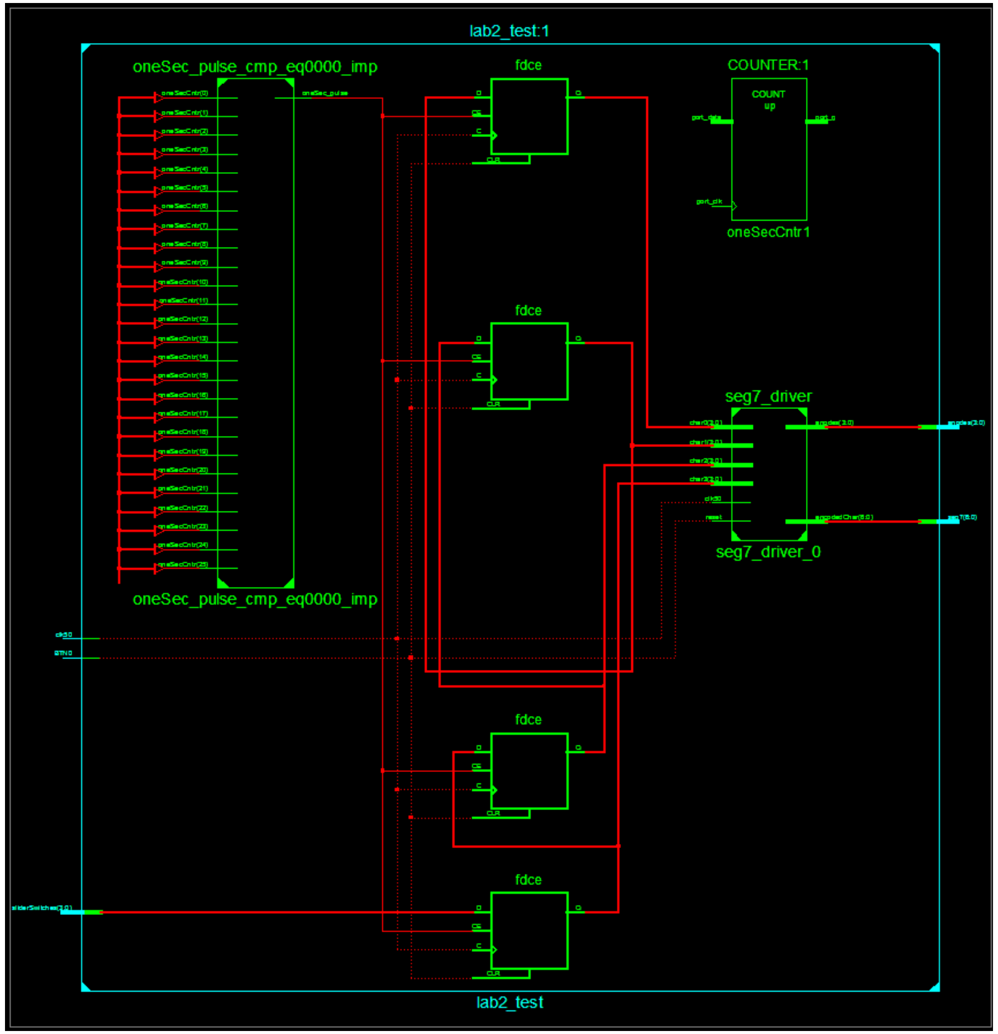
Synthesis Results – Four Processes

```
Device utilization summary:
---------------------------
Selected Device : 3s500efg320-5
 Number of Slices:                   41   out of   4656     0%
 Number of Slice Flip Flops:         62   out of   9312     0%
 Number of 4 input LUTs:             74   out of   9312     0%
 Number of IOs:                      17
 Number of bonded IOBs:              17   out of    232     7%
 Number of GCLKs:                     1   out of     24     4%
```

- What accounts for the 62 FFs in Lab 3?
- 4 FFs for the ___________
- 26 FFs for the _____________
- 16 FFs for the _____________
- 16 FFs for the _____________

4 anodes
1 Hz enable counter
1 KHz enable counter
4-bit, 4 deep shift register

Synthesis Results – Four Processes Continued



Synthesis Results – Single Process

```
Device utilization summary:
---------------------------
Selected Device : 3s500efg320-5
 Number of Slices:                   41  out of   4656      0%
 Number of Slice Flip Flops:         62  out of   9312      0%
 Number of 4 input LUTs:             74  out of   9312      0%
 Number of IOs:                      17
 Number of bonded IOBs:              17  out of    232      7%
 Number of GCLKs:                     1  out of     24      4%
```